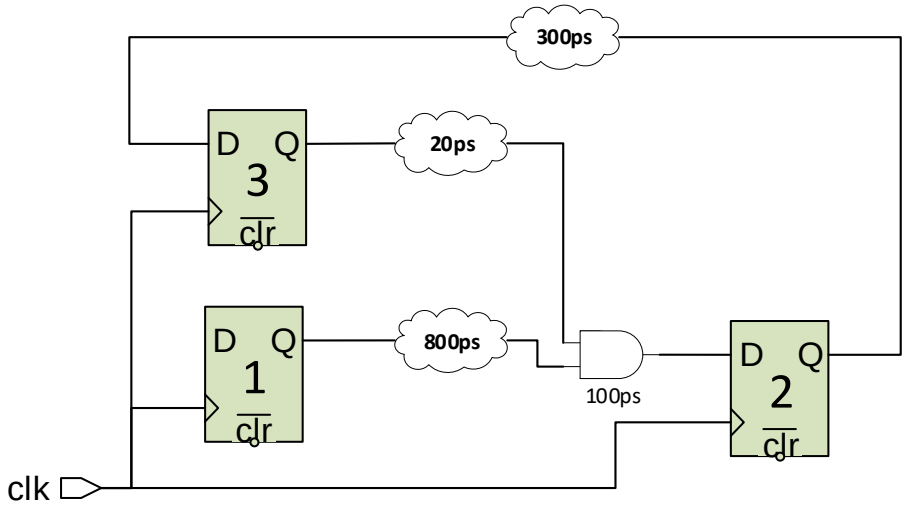
Advanced Logic Design – Lab Number 5

1. **(20 points) Timing**
   Given the following design:

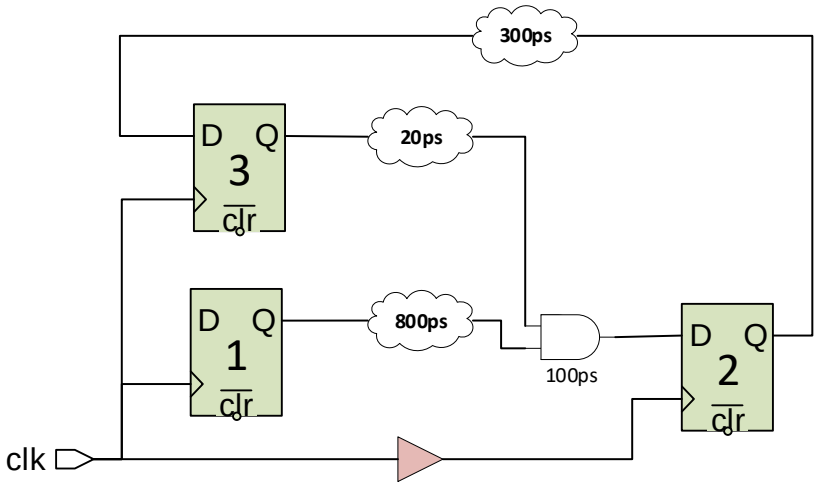


Fclk: 1GHz, ***Tsetup:*** 100ps, ***Thold:*** 150ps, ***Tcq (clock to q):*** 50ps

a. As you can see, there are 3 "flop to flop" timing paths in the design. Use the table below to answer the following questions:
   i. For each path, calculate the data arrival time and the data required time, for both setup and hold.
   ii. Which path has a violation? What is the violation type? What is the slack? Explain and include a wave diagram.

	FF1->FF2 setup	FF1->FF2 hold	FF3->FF2 setup	FF3->FF2 hold	FF2->FF3 setup	FF2->FF3 hold
Data arrival time						
Data required time						
Slack						
Meets timing						

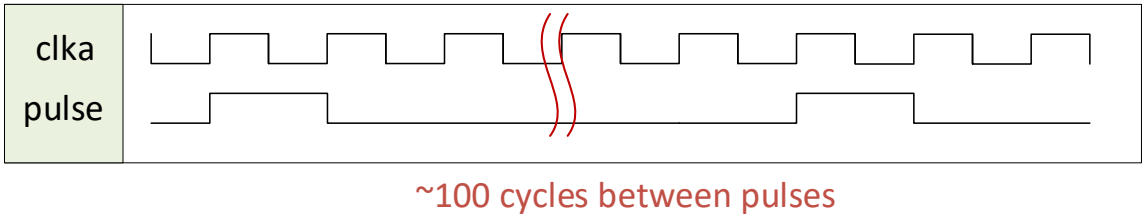
b. In order to fix the violation, one buffer was added as described below:



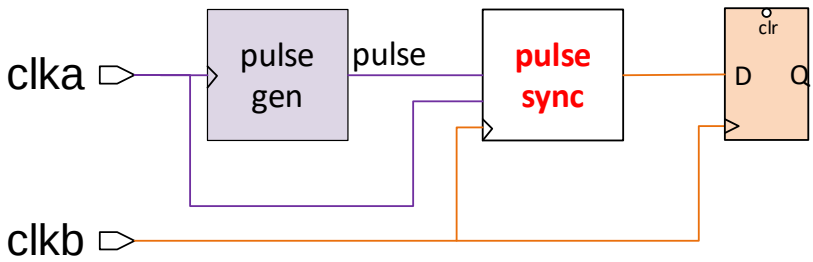
i. What should be the buffer delay in order to fix the timing violation?
ii. Update the table according to the design fix.
iii. Is there any new timing violation? how would you resolve it?

2. (20 points) Synchronization

pulse_gen is a block in **clka** domain, which generates 1 cycle pulses.
It is given that there are at least 100 clka cycles between pulse to pulse.



We need to sample that pulse with **clkb**.



You should implement the small module ***pulse_sync*** for each one of the following cases:

a. clkb is 2 times faster than clka, the phase between the clocks is unknown (**pulse_sync1**)
b. clka is 2 times faster than clkb, the phase between the clocks is unknown (**pulse_sync2**)
c. The relationship between clka and clkb is unknown (you can assume ratio less than 10) (**pulse_sync3**)

For each of the 3 cases above choose the most efficient synchronization method.

The interface definition of the ***pulse_sync***

Pin name	Direction	Clock domain	Description
clka	in	-	See 3 cases
resetb_a	in	clka	Active low async reset
clkb	in	-	See 3 cases
resetb_b	in	clkb	Active low async reset
pulse_in	in	clka	1 clka cycle
pulse_out	out	clkb	Synchronized pulse

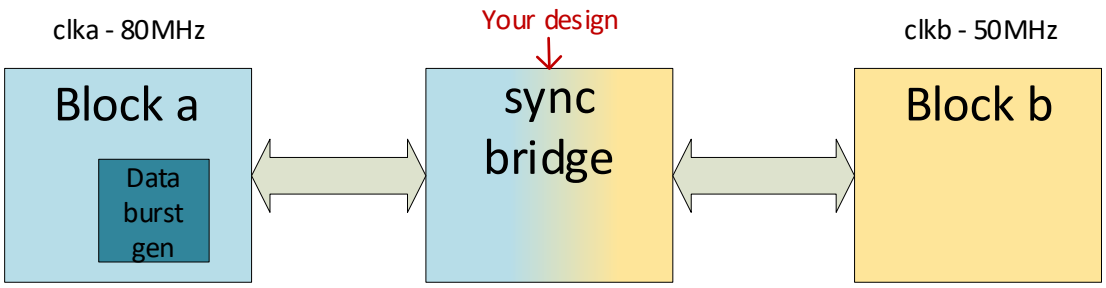
Submit for each case:

1. Schematic of the design
2. Wave diagram
3. Explain your considerations
4. SystemVerilog

Note: The SystemVerilog of all the 3 modules should be delivered **in the same file: pulse_sync.sv**

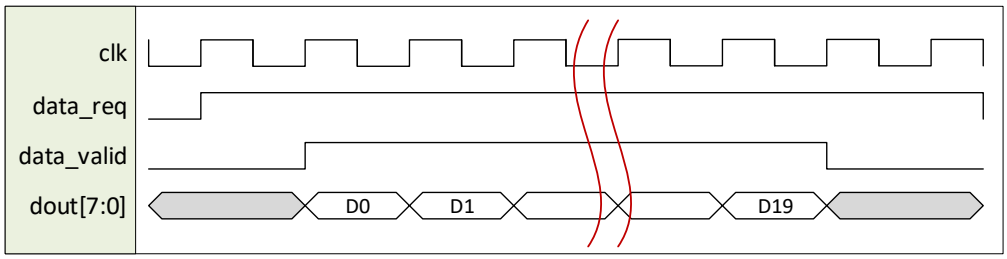
3. **(60 points) Synch bridge**

Design a module (*sync_bridge*) to synchronize between block **A** and block **B** as describes below. The *sync_bridge* should handle all CDC properly, to guarantee no metastability and no data loss.



Block A works at 80Mhz and Block B works at 50Mhz.

Block A generates a burst of 20 random numbers on *dout[7:0]* upon *data_req* is high.

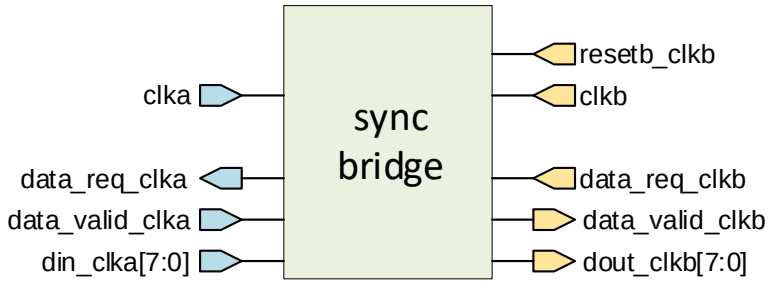


Block B drive the *data_req* high for one cycle and expect to get a burst of random data in response.

synch_bridge interface

Port name	Dir.	Clock domain	Description
clka	In	clka	80MHz clock
clkb	In	clkb	50MHz clock
resetb_clkb	In	clkb	Async active low reset, with positive edge synchronized with clkb
din_clka[7:0]	In	clka	Data from burst generator
data_req_clkb	In	clkb	Data request from block B (1 cycle)
data_valid_clka	In	clka	Data valid indication from block A
data_req_clka	Out	clka	Data request to block A
data_valid_clkb	Out	clkb	Data valid indication to block B
dout_clkb[7:0]	Out	clkb	Random data to block B

sync_bridge interface diagram



Implementation Guide

1) You probably noticed that an asynchronous FIFO is involved.
    a. Choose the FIFO depth and explain your calculations.
    b. Implement 2 **parameterized width** modules:
        i. bin2gray - Binary to Gray converter
        ii. gray2bin - Gray to Binary converter.
        You can find the algorithms in the web.
    c. Implement a standard asynchronous FIFO module according to the structure we learned in class.
2) Draw a diagram of your design - include the FIFO, the logic around, synchronizers etc. and the connectivity. (The FIFO's internal structure isn't needed)
3) Implement the top level design (sync_bridge.sv) in SystemVerilog, according to your diagram.
4) Write a testbench.
    a. Draw the testbench diagram.
    b. Implement separate drivers that mimic block A and block B behavior.
    c. Implement a monitor that collect the data and trigger the checker. For that purpose, you can use SystemVerilog queue data type.

```
integer queue1[$];                  // Queue declaration
queue1.push_back(data);             // Push data
popped_data = queue1.pop_front();   // Pop data to a variable.
```

    d. Implement a checker which verifies that all transmitted data are successfully arrived to block B.

- Choose wisely an efficient synchronization method for each CDC signal. Explain your considerations in the report.

- Double ff synchronizer, with parameterized width and uncertainty model is given (dff_sync.sv) – Note: this module is only for simulation it can't be synthesized.
- You can assume long delay between data transaction and next *data_req*. (next data request will be sent a safe time after the system settled down).
- You can specify any other reasonable assumption you have and design accordingly.

Submit:

1) **(10 points)** Detailed diagram of the sync_bridge
2) **(10 points)** Block diagram of the testbench
3) **(10 points)** Design report (FIFO depth and other considerations explanation)
4) SystemVerilog files:
   (20 points) sync_bridge.sv (include all the needed sub modules and functions)
   (10 points) sync_bridge_tb.sv (include all the needed sub modules and functions)

Good luck!